

Group 66 Specifications

Class design:

Interface

- Create Event Button
- Calendar (populates by inputs)
- Clickable Events

Event

- Event will represent one event with all its attributes the Attributes would include “name, date, time, location, event type, and optionally a notes section about that event”.

Calendar

- The calendar will have a list of all events and have different ways to show events

DateValidator

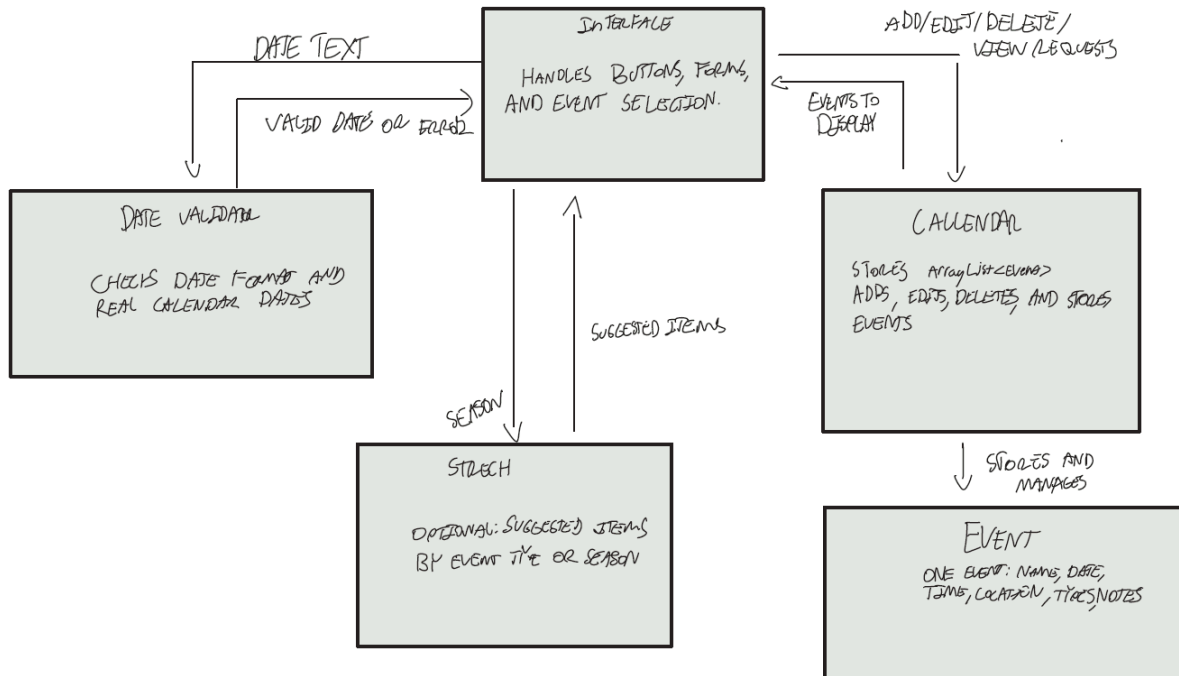
- This will check to make sure that the dates passed in are valid, and not fake or improperly formatted

Stretch-Goal-Class

- This would be a stretch goal class that would create a list of helpful items to bring to specific events based on required attributes.

System diagram:

HOKIE PLANNER 2.0



Data & state:

Interface

- Create Event Button
 - creatingEvent
 - Boolean - Checks to see if event form is open t or f
- Clickable Event
 - Clicking event
 - Stores if a user clicked an event can be displayed or edited. Also includes nothing being selectedCalendar (populates by inputs)
- Calendar
 - calenderPicture
 - Picture of the events in a month

Event

- Fields for Event
 - Name
 - String

- Date
 - LocalDate
- Time
 - LocalTime
- Location
 - String
- EventType
 - String
- Notes (optional)
 - String

Calendar

- Events
 - ArrayList<Event>
 - holds the events
- displayedMonth
 - int
 - Month being viewed, so you can look at specific events and times
- displayedYear
 - Int
 - Year being viewed, if you use a monthly view.

DateValidator

- DATE_FORMAT
 - DateTimeFormatter
 - Defines the accepted date format rules.

Stretch-Goal-Class

- itemsByEventType
 - HashMap<String, ArrayList<String>>
 - Stores each type as a key and list of suggested items to bring as its value. Most importantly we would implement the four seasons and give items to recommend for you to bring for each season or event such as birthday. For example if you listed birthday this would return a list containing, card and gift.

Method signatures:

Public class Event {

Fields...

Public Event(String name, String date, String time, String location, String eventType, String notes);

```

{
    this.name = name;
    this.date = date;
    this.time = time;
    this.location = location;
    this.eventType = eventType;
    this.notes = notes;
}

```

```

}

```

```

Public class Calendar {
    Public Calendar ( ){
    }
}

```

```

Public class DateValidator {

    Public DateValidator ( LocalDate date ) {
        this.date = date;
    }
}

```

```

Public class Stretch {
Public HashMap<String, ArrayList<String>>();
}

```

Where validation lives:

Bad input 1: Invalid date

- "33/FED/21"
- Caught by DateValidator.isValidDate();
 - Response: EventManager
 -
-
- ager rejects event = user re-prompt

Bad input 2: Wrong date format

- "DD/MM/YY" instead of "MM/DD/YY"
- Caught by DateValidator.parseDate();
 - Throws IllegalArgumentException = re- prompt user

Bad input 3: Entering in a Int for a String

- “76” instead of Vawter
- Caught by a try block
 - Throws IllegalArgumentException - re- prompt user

Test plan:

Event.addEvents(Event e)

Normal case:

Input: Event with valid date/time

Expected: Event added, list size increases by 1

Bad case:

Input: Event with date “33/FED/26.”

Expected: DateValidator rejects = addEvt returns false

DateValidator.parseDate()

Normal case:

Input: “08/31/2026”

Expected: LocalDate(2026, 8, 31)

Base case:

Input: “August 31st, 2026”

Expected: Expectation thrown

Calendar.getAll()

Normal case:

Input: 3 events with valid dates

Expected: Returned list sorted chronologically

Bad Case:

Input: one event with invalid time

Expected: Event not added = only valid events appear

Division of work:

We will assign different functions to different people so that each person's work does not conflict with each other's work . Also because the classes are not equal in work load so this assures everyone has a fair share of work.

Revised scope:

1. Added DateValidator class
 - a. A dedicated class for checking dates
2. Added method signatures
 - a. Added the signatures that we will be using
3. Refined our stretch goal
 - a. Instead of a multitude of events that have different pacing lists we decided only to use seasons.

AI Statement:

During the preparation of this assignment, the group used ChatGPT in the start of the deliverable to create a basic outline of our idea. After using this tool, we reviewed and edited the content as needed to ensure its accuracy and take full responsibility for the content in relation to grading. I understand that I am responsible for being able to complete this work without the use of assistance.